

RELAZIONE DEL PROGETTO

corso “BASI DI DATI”

(A.A 2023-2024)

Componenti del gruppo:

Gianluca Tondo - gianluca.tondo@studio.unibo.it - matr. 0001031952

Edoardo Casali - edoardo.casali2@studio.unibo.it - matr. 0001030816

Ana Maria Bala - anamaria.bala@studio.unibo.it - matr. 0001029919

ANALISI DEI REQUISITI

Testo completo delle specifiche sui dati

La piattaforma ESQL si basa sul database relazionale ESQldb.

Tutti gli utenti della piattaforma dispongono di: email, nome, cognome, eventuale recapito telefonico. Gli utenti sono divisi in due tipologie: docenti e studenti. I primi dispongono anche di: nome del dipartimento di appartenenza e nome del corso di cui sono titolari. I secondi dispongono anche di un campo anno di immatricolazione e di un codice alfanumerico di lunghezza pari a 16 caratteri. I docenti possono creare delle tabelle SQL (definite *tabelle di esercizio*, nel seguito): ogni tabella di esercizio dispone di nome, data di creazione, un campo num_righe. Inoltre, ogni tabella di esercizio dispone di un insieme di attributi: ogni attributo dispone di un nome, un tipo, e può far parte della chiave primaria della tabella di esercizio. Devono poter essere inseriti dai docenti anche gli vincoli di integrità tra attributi di diverse tabelle di esercizio. In aggiunta, ogni docente può creare dei test: ogni test dispone di un titolo univoco, una data di creazione ed un'eventuale foto. Ogni test può includere una serie di quesiti: ogni quesito dispone di un numero progressivo (univoco, ma solo all'interno di uno specifico test), un livello di difficoltà (campo enum con valori: Basso, Medio, Alto), un campo descrizione, un campo #numrisposte (ridondanza concettuale, vedere specifiche sotto) e fa riferimento ad una o più tabelle di esercizio tra quelle create dal docente. I quesiti possono appartenere esclusivamente a due categorie: quesiti a risposta chiusa o quesiti di codice. Nel primo caso, il quesito dispone di una serie di opzioni di risposta: ogni opzione dispone di una numerazione (univoca, ma solo all'interno di uno specifico quesito) ed un campo testo. Nel secondo caso, il quesito dispone di una o più soluzioni (sketch di codice SQL che implementano correttamente quanto richiesto dalla descrizione del quesito). Ogni test dispone di un campo booleano VisualizzaRisposte: se settato a True, le risposte dei quesiti diventano visibili agli studenti, altrimenti restano nascoste. Gli studenti possono svolgere un test, inserendo una o più risposte per ciascun quesito. Si vuole tenere traccia del completamento del test, ossia: data di inserimento della prima risposta (su scala temporale), data di inserimento dell'ultima risposta (su scala temporale), stato (campo enum con tre valori: Aperto, InCompletamento, Concluso). Nel caso di quesiti a risposta chiusa, la risposta consiste nell'opzione scelta tra quelle disponibili. Nel caso di quesiti di codice, la risposta consiste in un campo testo (codice SQL che risolve l'esercizio). E' prevista la possibilità per lo studente di sottoporre più risposte per lo stesso quesito, in istanti diversi di tempo, ma solo se il test non è stato Concluso. Ogni risposta dispone di un campo esito, che può valere True o False a seconda che la risposta fornita dallo studente coincida con quella inserita dal docente (nel caso di quesiti a risposta chiusa) o che la risposta fornita dallo studente produca lo stesso output di quella inserita dal docente (nel caso di quesiti di codice). Infine, è prevista la possibilità di inviare messaggi nella piattaforma. Ogni messaggio dispone di un titolo, un campo testo, una data di inserimento, e fa riferimento ad uno specifico test. Il messaggio può essere inviato da un docente: in tal caso, il messaggio viene ricevuto da tutti gli studenti.

Viceversa, un messaggio può essere inviato da uno studente: in tal caso, il destinatario è uno specifico docente.

Infine, si vuole tenere traccia di tutti gli eventi che occorrono nella piattaforma, relativamente all'inserimento di nuovi dati (es. nuovi utenti, nuovi test, nuovi quesiti, etc). Tali eventi vanno inseriti, sotto forma di messaggi di testo, all'interno di un log, implementato in un'apposita collezione MongoDB.

Decomposizione del testo in frasi omogenee

Frase relative agli utenti:

Tutti gli utenti hanno un'email, un nome, un cognome e possono fornire un recapito telefonico. Gli utenti si dividono in due categorie principali: docenti e studenti. I docenti, oltre ai dati personali comuni, hanno informazioni aggiuntive come il nome del dipartimento e del corso a cui appartengono. Gli studenti dispongono di un anno di immatricolazione e un codice alfanumerico di 16 caratteri.

Frase relative alle tabelle

Ogni tabella di esercizio ha un nome, una data di creazione e un campo che tiene traccia del numero di righe. Ogni tabella contiene anche attributi con un nome, un tipo, e la possibilità di far parte della chiave primaria. I docenti possono anche definire vincoli di integrità tra attributi appartenenti a tabelle diverse.

Frase relative ai test e quesiti

Ogni test ha un titolo univoco, una data di creazione e, facoltativamente, una foto. Ogni test include quesiti con un numero progressivo (unico per ciascun test), un livello di difficoltà, una descrizione, e può riferirsi a una o più tabelle di esercizio create dal docente. I quesiti possono essere a risposta chiusa, con opzioni numerate e testi specifici, o quesiti di codice, con una o più soluzioni SQL.

I test hanno un campo booleano chiamato VisualizzaRisposte, che se impostato su True permette agli studenti di visualizzare le risposte. Gli studenti possono rispondere ai test inserendo risposte ai quesiti. Per ogni quesito, si tiene traccia del completamento del test, con

la data della prima risposta, dell'ultima risposta, e lo stato che può essere Aperto, InCompletamento o Concluso.

Frase relative ai messaggi

La piattaforma supporta l'invio di messaggi. Un messaggio può essere inviato da un docente a tutti gli studenti o da uno studente a un docente specifico.

Glossario dei termini

Termine	Descrizione	Sinonimi	Collegamenti
UTENTE	Rappresenta un utente generico della piattaforma con email, nome, cognome, e recapito telefonico.		Collegato a DOCENTE e STUDENTE, invia MESSAGGI, partecipa a TEST e invia RISPOSTE.
DOCENTE	Un utente con il ruolo di docente, affiliato a un dipartimento e titolare di un corso.	Professore	Crea TABELLA SQL, TEST, QUESITI, invia MESSAGGI_DOCENTE a STUDENTE.
STUDENTE	Un utente con il ruolo di studente, identificato tramite anno di immatricolazione e un codice alfanumerico.	Allievo	Partecipa a TEST, invia MESSAGGI_STUDENTE a DOCENTE, fornisce RISPOSTE a QUESITI.
MESSAGGIO	Comunicazione tra docente e studenti, include titolo, testo, e data di inserimento.	Nota	MESSAGGIO_DOCENTE o MESSAGGIO_STUDENTE è collegato a DOCENTE, STUDENTE, e TEST.
TEST	Valutazione composta da quesiti, con titolo univoco, data di creazione e visibilità delle risposte.	Esame	Include QUESITI, può essere svolto da uno STUDENTE con RISPOSTE.
TABELLA SQL	Una tabella creata dal docente, contiene nome, data di creazione e numero di righe.		Collegata agli ATTRIBUTI, associata a QUESITI.
ATTRIBUTO	Campo all'interno di una tabella SQL, ha un nome, tipo, e può far parte della chiave primaria.	Colonna	Associato a TABELLA SQL.

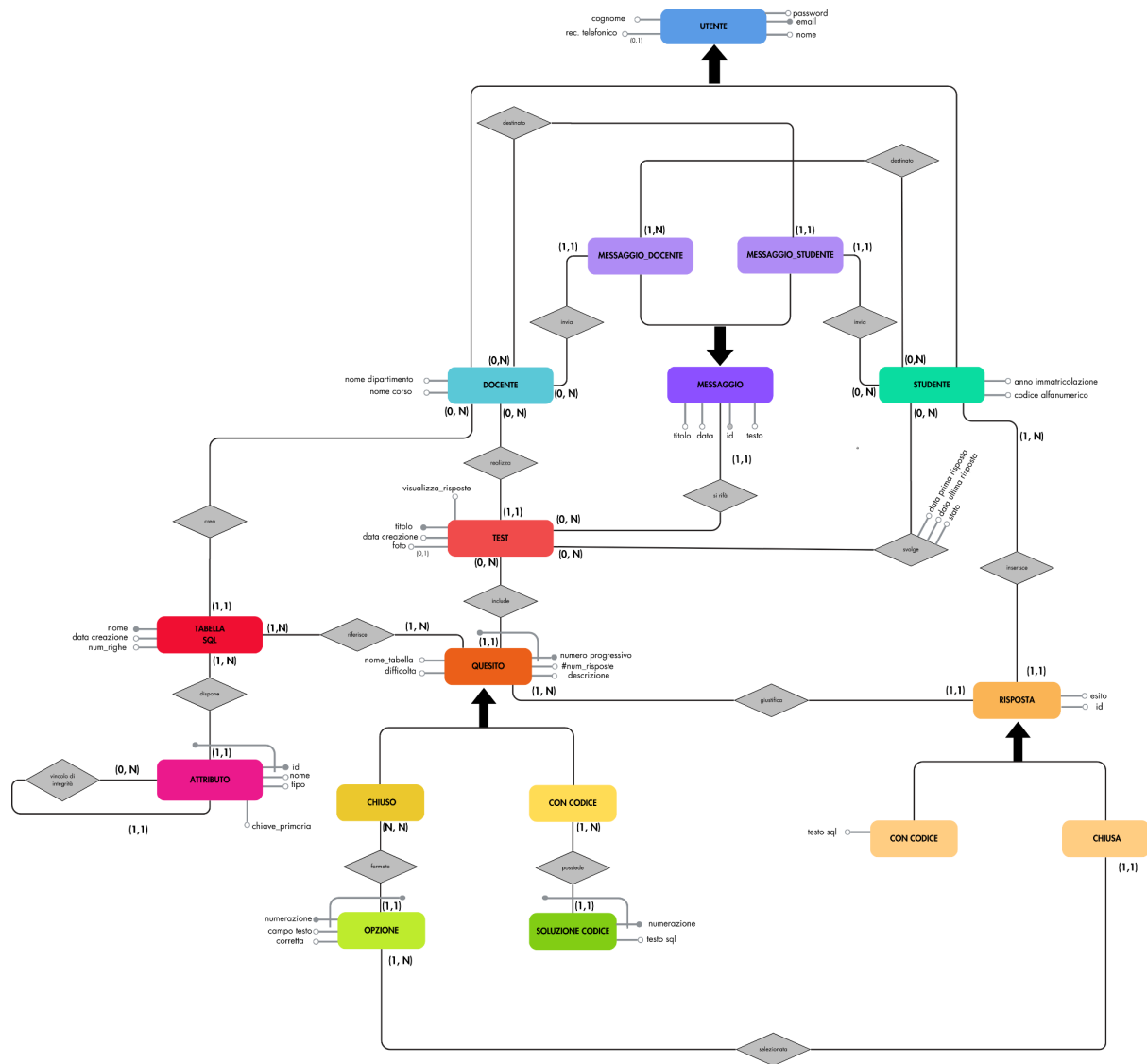
QUESITO	Domanda all'interno di un test, ha un numero progressivo, livello di difficoltà, e descrizione.	Domanda	Collegato a TEST, associato a TABELLA SQL, può essere a risposta CHIUSA o con CON CODICE.
OPZIONE	Opzione di risposta per un quesito chiuso, con numerazione e testo.	Scelta	Collegato a QUESITO chiuso.
SOLUZIONE CODICE	Soluzione per un quesito con codice SQL, ha numerazione e testo.	Risposta	Collegato a QUESITO con codice.
RISPOSTA	Risposta data dallo studente a un quesito, con esito True/False a seconda della correttezza.		Collegata a STUDENTE, TEST, e QUESITO.

Operazioni sui dati

- **OPERAZIONE 1:** Inserire un nuovo utente (docente o studente).
- **OPERAZIONE 2:** Inserire una nuova tabella di esercizio.
- **OPERAZIONE 3:** Aggiungere attributi a una tabella di esercizio.
- **OPERAZIONE 4:** Definire vincoli di integrità tra tabelle di esercizio.
- **OPERAZIONE 5:** Creare un nuovo test.
- **OPERAZIONE 6:** Inserire un quesito in un test.
- **OPERAZIONE 7:** Aggiungere opzioni di risposta a un quesito chiuso.
- **OPERAZIONE 8:** Inserire una soluzione per un quesito di codice.
- **OPERAZIONE 9:** Registrare una risposta di uno studente per un test.
- **OPERAZIONE 10:** Verificare l'esito di una risposta fornita dallo studente.
- **OPERAZIONE 11:** Aggiornare lo stato di un test in base alle risposte fornite.
- **OPERAZIONE 12:** Visualizzare le risposte corrette di un test, se consentito.
- **OPERAZIONE 13:** Inviare un messaggio da un docente a tutti gli studenti.
- **OPERAZIONE 14:** Inviare un messaggio da uno studente a un docente.
- **OPERAZIONE 15:** Visualizzare i test completati da uno studente.
- **OPERAZIONE 16:** Visualizzare le statistiche di completamento dei test e delle risposte corrette per gli studenti

PROGETTAZIONE CONCETTUALE

Diagramma E-R



Docente: email_docente(pk), password, nome, cognome, nome_dipartimento, nome_corso

Studente: email_studente(pk), password, nome, cognome, anno_immatricolazione, codice_alfanumerico

Telefono_Docente: email_docente(fk)(pk), rec_telefonico(pk)

Telefono_Studente: email_studente(fk)(pk), rec_telefonico(pk)

TabellaSQL: nome_Tabella(pk), email_docente(fk)(pk), data_creazione, num_righe.

Attributo: id(pk), nome_TabellaSQL(fk), nome, tipo, chiave_primaria

VincoloDiIntegrità: id_attributo_referenziante, id_attributo_referenziato

Test: titolo_Test(pk), email_docente(fk), data_creazione, visualizza_risposte

Foto_Test: titolo_Test(pk)(fk), url_Foto(pk)

Quesito: numero_quesito(pk), titolo_Test(pk)(fk), difficoltà, descrizione, tipo, num_risposte.

Riferisce (Quesito - TabellaSQL): numero_quesito(pk), titolo_Test(pk)(fk), nome_Tabella(pk)(fk)

Opzione: numero_opzione(pk), numero_quesito(pk)(fk), titolo_Test(pk)(fk), campo_testo, corretta.

Soluzione_Codice: numero_soluzione(pk), numero_quesito(pk)(fk), titolo_Test(pk)(fk), testo_SQL.

Svolge (Studente - Test): email_studente(fk)(pk), titolo_Test(pk)(fk), data_ultima_risposta, data_prima_risposta, stato

RispostaChiusa: id(pk), email_studente(fk), numero_opzione(fk), numero_quesito(fk), titolo_Test(fk), esito

RispostaCodice: id(pk), email_studente(fk), esito, testo_sql

Messaggio_Studente: id(pk), email_studente(fk), email_docente(fk), titolo_Test(fk), data, titolo, testo

Messaggio_Docente: id(pk), email_docente(fk), titolo_Test(fk), data, titolo, testo

Dizionario delle Entità Relazione

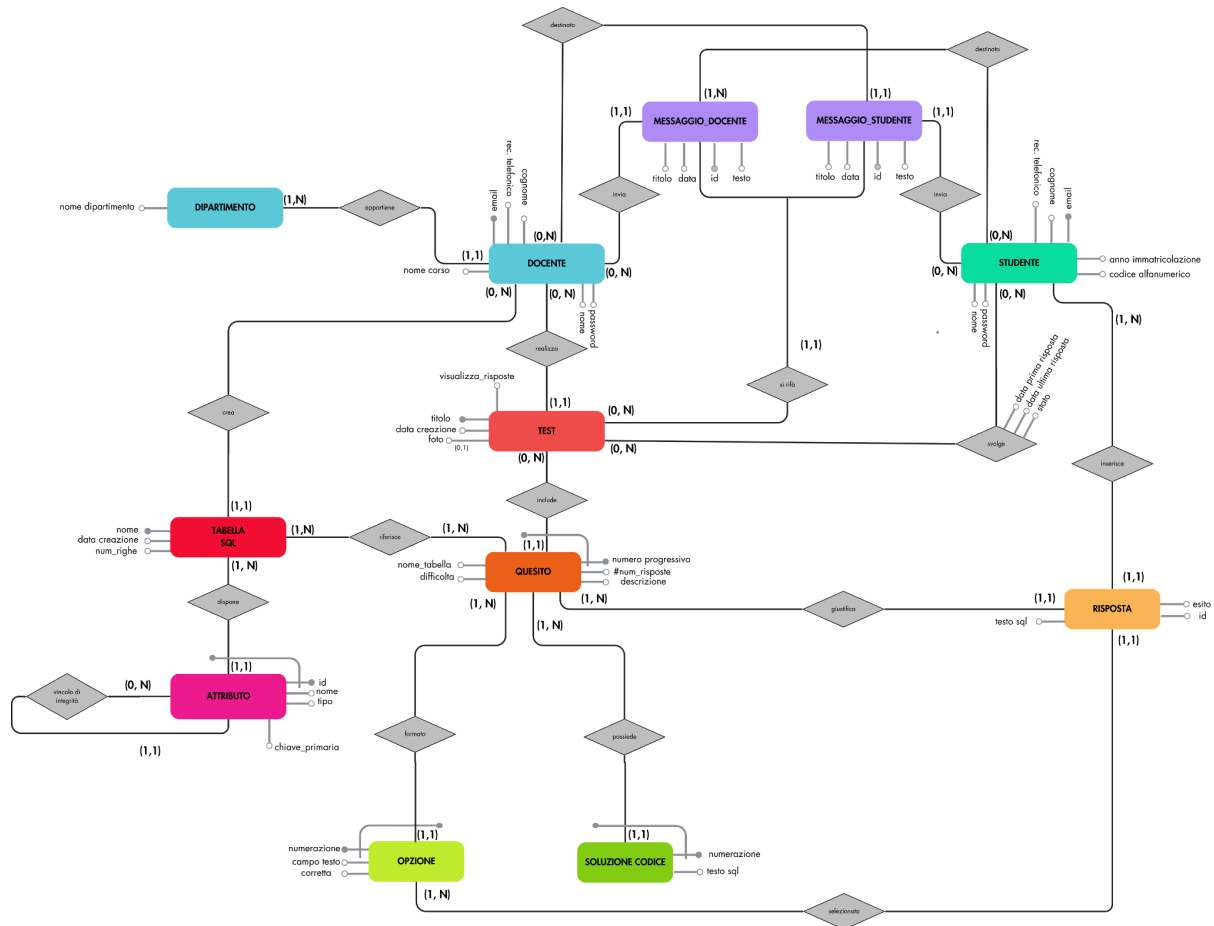
Dizionario delle entità			
Entità	Descrizione	Attributi	Identificatore
Utente	Persona (in senso generale) che accede alla piattaforma ESQL	email, nome, cognome, password, rec. telefonico	email
Docente	Insegnanti	nome dipartimento, nome corso	email
Studente	Colui che risponde alle domande dei test	anno_immatricolazione, codice_alfanumerico	email
Messaggio	Comunicazione tra studente e docente	id, titolo, data, test	id
Messaggio docente	Messaggio da parte di un docente		id
Messaggio studente	Messaggio da parte di uno studente		id
Test	Prova con domande	titolo, visualizza_risposte, data_creazione, foto	titolo
Tabella SQL	Righe e colonne che contengono dati	nome, data_creazione, num_righe	nome
Attributo	Record di tabella	id, nome, tipo	id
Quesito	Domanda	numero_progressivo, num_risposte, descrizione, nome_tabella, difficolta	numero_progr essivo
Chiuso	Domanda a risposta chiusa		numero_progr essivo
Con codice	Domanda aperta		numero_progr essivo
Opzione	Risposta	numerazione, campo_testo, corretta	numerazione
Soluzione codice	Query corretta	numerazione, testo_sql	numeraizone
Risposta	Risposta dello studente	id, esito	id
Con codice	Query dello studente	testo_sql	id
Chiusa	opzione scelta dallo studente		id
Destinato	Destinazione messaggio	Utente - Messaggio	
invia	utente via messaggio	Utente - Messaggio	
realizza	docente realizza i test	Docente - Test	
crea	docente crea tabelle sql	Docente - Tabella SQL	
dispone	la tabelle sql dispone di attributi	Tabella SQL - Attributo	
vincolo di	attributo vincolo di integrità (fk)	Attributo - Attributo	

integrità			
referisce	quesito si riferisce a tabella/e SQL	Quesito - Tabella SQL	
include	test include quesito	Test - Quesito	
si rifa	messaggio si rifà ad un test	Messaggio - Test	
svolge	Studente svolge test	Studente - Test	
inserisce	studente inserisce risposta	Studente - Risposta	
consiste	la risposta consiste per un determinato quesito	Risposta - Quesito	
formato	l'opzione più essere formata da quesiti chiusi	Opzione - Chiuso	
possiede	il quesito con codice può possedere soluzioni di codice	Soluzione codice - Quesito	
selezionata	La risposta chiusa è selezionata da un'opzione	Chiusa - Opzione	

Tavola delle Business Rules

Regole di vincolo
1. Ogni utente (sia studente che docente) deve disporre di un'email, nome, cognome e, facoltativamente, di un recapito telefonico.
2. Gli studenti devono avere un campo per l'anno di immatricolazione e un codice alfanumerico di 16 caratteri.
3. I docenti devono avere un campo per il nome del dipartimento di appartenenza e il nome del corso di cui sono titolari.
4. I docenti possono creare delle tabelle SQL (dette "tabelle di esercizio"), che devono includere nome, data di creazione e numero di righe (num_righe).
5. Ogni tabella di esercizio deve avere uno o più attributi, ciascuno con un nome, un tipo, e con la possibilità di far parte della chiave primaria.
6. I docenti devono poter inserire vincoli di integrità tra attributi di diverse tabelle di esercizio.
7. Ogni test deve avere un titolo univoco e una data di creazione.
8. I quesiti devono essere collegati a una o più tabelle di esercizio create dal docente.
9. I quesiti devono appartenere a due categorie: a risposta chiusa o di codice.
10. I quesiti a risposta chiusa devono avere una serie di opzioni di risposta, ognuna con una numerazione univoca (all'interno di uno specifico quesito) e un campo testo.
11. I quesiti di codice devono avere una o più soluzioni sotto forma di sketch di codice SQL.
12. Il campo booleano VisualizzaRisposte deve determinare se le risposte sono visibili o nascoste agli studenti.
13. Gli studenti possono inserire una o più risposte per ciascun quesito.
14. Gli studenti possono sottoporre più risposte per lo stesso quesito, ma solo finché il test non è stato concluso.
15. I messaggi inviati da un docente sono ricevuti da tutti gli studenti; quelli inviati da uno studente sono ricevuti da un docente specifico.
Regole di derivazione
1. Il completamento del test deve essere tracciato con data di inserimento della prima risposta, data di inserimento dell'ultima risposta e stato (enum: Aperto, InCompletamento, Concluso).
2. Per i quesiti a risposta chiusa, la risposta dello studente deve corrispondere a un'opzione scelta tra quelle disponibili.
3. Per i quesiti di codice, la risposta deve essere un campo testo contenente il codice SQL che risolve l'esercizio.
4. Ogni risposta deve avere un campo esito (True/False) in base alla corrispondenza con la risposta del docente (quesiti a risposta chiusa) o in base al fatto che la risposta produca lo stesso output della soluzione fornita dal docente.

Ristrutturazione dello schema concettuale



Analisi delle ridondanze e tavola dei volumi

Concetto	Tipo	Volume
Utente	Entità	50
Quesito	Entità	20
Risposta	Entità	10

Valutare se la ridondanza campo **#num_risposte** relativo ad un quesito debba essere tenuta o eliminata sulla base di queste operazioni:

- Aggiungere una nuova risposta ad un quesito esistente (10 volte/mese, interattiva)
- Rimuovere un quesito e tutte le risposte ottenute (2 volte/mese, batch)
- Visualizzare tutti gli utenti presenti nella piattaforma (1 volta/mese, batch)
- Contare il numero di risposte per ciascun quesito presente nella piattaforma (2 volte/mese, interattiva)

Operazione	Tipo	Frequenza al mese
Operazione 1	Interattiva	10
Operazione 2	Batch	2
Operazione 3	Batch	1
Operazione 4	Interattiva	2

Coefficienti per l'analisi: $w_I = 1$, $w_B = 0.5$, $a = 2$

Tabella dei volumi: 10 risposte per quesito, 20 quesiti, 50 utenti

Con costo:

$$c(O_T) = f(O_T) \cdot w_T \cdot (\alpha \cdot NC_{write} + NC_{read})$$

Analisi dello schema senza ridondanze

OPERAZIONE 1

Abbiamo 2 accessi (Risposta e Giustifica), quindi $NC_{write} = 1 + 1$

$$Costo(operazione1) = 10 \times 1 \times (2 \times 2 + 0) = 40$$

OPERAZIONE 2

Abbiamo 1 accesso a Quesito, 10 a Giustifica e 10 a Risposta, quindi $NC_{write} = 1 + 10 + 10$

$$Costo(operazione2) = 2 \times 0.5 \times (2 \times 11 + 0) = 22$$

OPERAZIONE 3

Abbiamo 50 accessi a Utente, quindi $NC_{read} = 50$

$$Costo(operazione2) = 1 \times 0.5 \times (2 \times 0 + 50) = 25$$

OPERAZIONE 4

Abbiamo 20x10 accessi, quindi $NC_{read} = 200$

$$Costo(operazione2) = 2 \times 1 \times (2 \times 0 + 200) = 400$$

$$Costo\ complessivo\ C = 40 + 22 + 25 + 400 = 487$$

Analisi dello schema con ridondanze

OPERAZIONE 1

Abbiamo 3 accessi (Risposta, Giustifica e Quesito), quindi $NC_{write} = 1 + 1 + 1$

$$Costo(operazione1) = 10 \times 1 \times (2 \times 3 + 0) = 60$$

OPERAZIONE 2 (UGUALE)

Abbiamo 1 accesso a Quesito, 10 a Giustifica e 10 a Risposta, quindi $NC_{write} = 1 + 10 + 10$

$$Costo(operazione2) = 2 \times 0.5 \times (2 \times 11 + 0) = 22$$

OPERAZIONE 3 (UGUALE)

Abbiamo 50 accessi a Utente, quindi $NC_{read} = 50$

$$Costo(operazione2) = 1 \times 0.5 \times (2 \times 0 + 50) = 25$$

OPERAZIONE 4

Abbiamo 20, quindi $NC_{read} = 20$

$$Costo(operazione2) = 2 \times 1 \times (2 \times 0 + 20) = 40$$

$$Costo\ complessivo\ C_{rid} = 40 + 22 + 25 + 400 = 147$$

Memoria occupata

Memoria senza ridondanza = X byte

Memoria con ridondanza = $X + (4 \times 20)$ byte = $X + 80$ byte

Conclusioni

Rapporto = $C / C_{rid} = 487 / 147 = 3,31$

Con la ridondanza viene migliorato lo speedup delle operazioni di 3,31 volte, introducendo però un overhead di memoria di 80 byte, per cui decidiamo di usare la ridondanza concettuale *num_risposte*.

Lista delle tabelle con i vincoli di chiave

Docente

- Chiave Primaria (PK): email_docente

Studente

- Chiave Primaria (PK): email_studente

Telefono_Docente

- Chiave Primaria Composta (PK): email_docente, rec_telefonico
- Chiave Esterna (FK): email_docente fa riferimento a Docente(email_docente)

Telefono_Studente

- Chiave Primaria Composta (PK): email_studente, rec_telefonico
- Chiave Esterna (FK): email_studente fa riferimento a Studente(email_studente)

TabellaSQL

- Chiave Primaria Composta (PK): nome_Tabella, email_docente
- Chiave Esterna (FK): email_docente fa riferimento a Docente(email_docente)

Attributo

- Chiave Primaria (PK): id
- Chiave Esterna (FK): nome_TabellaSQL fa riferimento a TabellaSQL(nome_Tabella)

VincoloDiIntegrità

- **Chiave Primaria Composta (PK):** id_attributo_referenziante, id_attributo_referenziato
- **Chiave Esterna (FK):** id_attributo_referenziante fa riferimento a Attributo(id)
- **Chiave Esterna (FK):** id_attributo_referenziato fa riferimento a Attributo(id)

Test

- **Chiave Primaria (PK):** titolo_Test
- **Chiave Esterna (FK):** email_docente fa riferimento a Docente(email_docente)

Foto_Test

- **Chiave Primaria Composta (PK):** titolo_Test, url_Foto
- **Chiave Esterna (FK):** titolo_Test fa riferimento a Test(titolo_Test)

Quesito

- **Chiave Primaria Composta (PK):** numero_quesito, titolo_Test
- **Chiave Esterna (FK):** titolo_Test fa riferimento a Test(titolo_Test)

Riferisce

- **Chiave Primaria Composta (PK):** numero_quesito, titolo_Test, nome_Tabella
- **Chiave Esterna (FK):** titolo_Test fa riferimento a Test(titolo_Test)
- **Chiave Esterna (FK):** nome_Tabella fa riferimento a TabellaSQL(nome_Tabella)

Opzione

- **Chiave Primaria Composta (PK):** numero_opzione, numero_quesito, titolo_Test
- **Chiave Esterna (FK):** numero_quesito, titolo_Test fa riferimento a Quesito(numero_quesito, titolo_Test)

Soluzione_Codice

- **Chiave Primaria Composta (PK):** numero_soluzione, numero_quesito, titolo_Test
- **Chiave Esterna (FK):** numero_quesito, titolo_Test fa riferimento a Quesito(numero_quesito, titolo_Test)

Svolge (Studente - Test)

- **Chiave Primaria Composta (PK):** email_studente, titolo_Test
- **Chiave Esterna (FK):** email_studente fa riferimento a Studente(email_studente)
- **Chiave Esterna (FK):** titolo_Test fa riferimento a Test(titolo_Test)

RispostaChiusa

- **Chiave Primaria (PK):** id
- **Chiave Esterna (FK):** email_studente fa riferimento a Studente(email_studente)
- **Chiave Esterna (FK):** numero_quesito, titolo_Test fa riferimento a Quesito(numero_quesito, titolo_Test)
- **Chiave Esterna (FK):** numero_opzione fa riferimento a Opzione(numero_opzione, numero_quesito, titolo_Test)

RispostaCodice

- **Chiave Primaria (PK):** id
- **Chiave Esterna (FK):** email_studente fa riferimento a Studente(email_studente)
- **Chiave Esterna (FK):** numero_quesito, titolo_Test fa riferimento a Quesito(numero_quesito, titolo_Test)

Messaggio_Studente

- **Chiave Primaria (PK):** id
- **Chiave Esterna (FK):** email_studente fa riferimento a Studente(email_studente)
- **Chiave Esterna (FK):** email_docente fa riferimento a Docente(email_docente)
- **Chiave Esterna (FK):** titolo_Test fa riferimento a Test(titolo_Test)

Messaggio_Docente

- **Chiave Primaria (PK):** id
- **Chiave Esterna (FK):** email_docente fa riferimento a Docente(email_docente)
- **Chiave Esterna (FK):** titolo_Test fa riferimento a Test(titolo_Test)

Lista delle tabelle con i vincoli di chiave

1. **Docente - Test**
 - Un docente può creare più test. Il vincolo di chiave esterna tra Test(email_docente) e Docente(email_docente) rappresenta questa relazione.
2. **Studente - Test (Svolge)**
 - Un test può essere svolto da più studenti. La tabella Svolge tiene traccia di quali studenti hanno svolto quali test. Il vincolo di chiave esterna tra Svolge(email_studente) e Studente(email_studente) e tra Svolge(titolo_Test) e Test(titolo_Test) rappresenta questa relazione.
3. **Test - Quesito**
 - Ogni test può includere uno o più quesiti. Il vincolo di chiave esterna tra Quesito(titolo_Test) e Test(titolo_Test) rappresenta questa relazione.

4. Quesito - TabellaSQL (Riferisce)

- Ogni quesito può fare riferimento a una o più tabelle SQL create dal docente. La relazione è rappresentata dalla tabella Riferisce, che usa chiavi esterne verso Quesito e TabellaSQL.

5. Quesito - Opzione

- Per i quesiti a risposta chiusa, ci possono essere diverse opzioni di risposta. La tabella Opzione tiene traccia di queste opzioni e ha vincoli di chiave esterna verso Quesito.

6. Quesito - Soluzione_Codice

- I quesiti di codice possono avere una o più soluzioni. La tabella Soluzione_Codice fa riferimento a Quesito tramite vincoli di chiave esterna.

7. Studente - RispostaChiusa

- Gli studenti possono rispondere a quesiti a risposta chiusa scegliendo tra le opzioni disponibili. La tabella RispostaChiusa tiene traccia delle risposte degli studenti, con vincoli di chiave esterna verso Studente, Quesito e Opzione.

8. Studente - RispostaCodice

- Gli studenti possono rispondere ai quesiti di codice con una soluzione SQL. La tabella RispostaCodice tiene traccia delle risposte con vincoli di chiave esterna verso Studente e Quesito.

9. Docente - Messaggio_Docente

- I docenti possono inviare messaggi agli studenti riguardanti i test. La tabella Messaggio_Docente ha chiavi esterne verso Docente e Test.

10. Studente - Messaggio_Studente

- Gli studenti possono inviare messaggi ai docenti. La tabella Messaggio_Studente tiene traccia di questi messaggi, con chiavi esterne verso Studente, Docente e Test.

NORMALIZZAZIONE

Seconda Forma Normale

Per essere in 2NF, in una tabella non devono esserci dipendenze parziali, ossia nessun attributo non chiave deve dipendere solo da una parte della chiave primaria.

Nelle tabelle, non ci sono dipendenze parziali. Ad esempio:

- Nella tabella Quesito, la chiave primaria è composta da *numero_quesito* e *titolo_test*, e tutti gli attributi non chiave (come *difficoltà*, *descrizione*, *tipo*) dipendono dall'intera chiave composta.
- Nella tabella Opzione, la chiave primaria è composta da *numero_opzione*, *numero_quesito*, e *titolo_test*, e non ci sono dipendenze parziali.

Terza Forma Normale

Per essere in 3NF, una tabella deve essere in 2NF e non devono esserci dipendenze transitive, ossia nessun attributo non chiave deve dipendere da altri attributi non chiave.

Nelle tabelle, non ci sono dipendenze transitive. Ogni attributo non chiave dipende direttamente dalla chiave primaria.

Forma Normale di Boyce-Codd (BCNF)

In BCNF, ogni dipendenza funzionale $X \rightarrow Y$ deve avere che X è una superchiave, quindi deve contenere un attributo candidato per essere la chiave primaria.

Le tabelle rispettano questa forma, poiché tutte le dipendenze funzionali rispettano questa regola. Ad esempio:

- In Docente, l'attributo *email_docente* è una superchiave e determina tutti gli altri attributi.
- In Svolge (Studente - Test), la chiave composta da *email_studente* e *titolo_test* determina univocamente tutti gli altri attributi.

DESCRIZIONE DELLE FUNZIONALITÀ DELL'APPLICAZIONE WEB

L'applicazione web ESQL permette supportare la didattica sulla programmazione SQL. Gli utenti (**docenti** e **studenti**) dispongono di funzionalità specifiche in base al ruolo.

Accesso all'applicazione

Per avviare l'applicazione, è necessario accedere al file "index.php", che presenta la pagina di login. Gli utenti possono autenticarsi o registrarsi come **docente** o **studente**. In base al tipo di utente, l'applicazione reindirizza alla rispettiva **home page** (home_docente o home_studente).

Funzionalità per i docenti

Nella **home del docente**, sono disponibili quattro sezioni principali:

1. **Gestione tabelle:** Il docente può creare nuove tabelle SQL utilizzando un editor SQL dedicato. Dopo aver creato una tabella, è possibile popolarla tramite un form apposito e visualizzarne i dati.
2. **Gestione test:** Il docente può creare nuovi test definendo un titolo e, facoltativamente, allegando una foto. Successivamente, è possibile aggiungere quesiti di due tipi: **quesiti a risposta chiusa**, che richiedono la definizione di quattro opzioni (di cui una corretta), e **quesiti di codice**, per cui il docente può testare e inserire soluzioni SQL nell'editor. Una volta completato, il test viene salvato.. Il docente può visualizzare i test esistenti e modificare lo stato del campo "Visualizza Risposte" tramite un apposito pulsante.
3. **Gestione messaggi:** Il docente può inviare messaggi a tutti gli studenti di un test specifico, oppure leggere i messaggi inviati dagli studenti.
4. **Visualizzazione delle statistiche:** In questa sezione, il docente può visualizzare statistiche dettagliate relative ai test eseguiti dagli studenti.

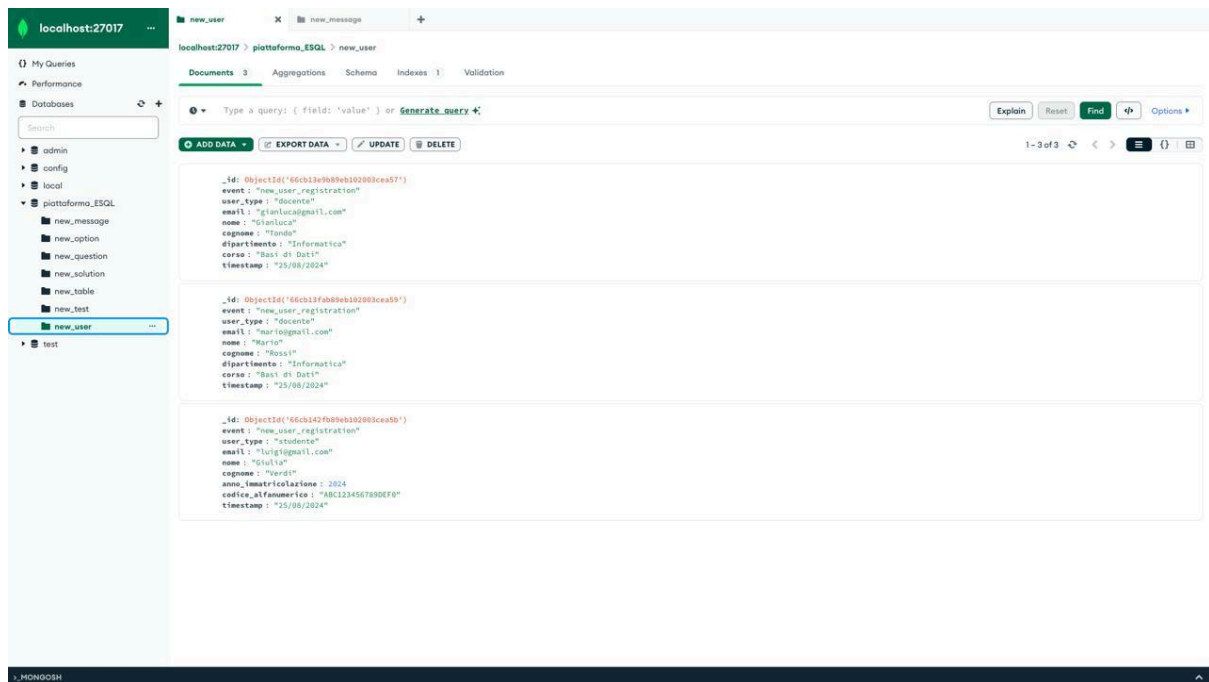
Funzionalità per gli studenti

Nella **home dello studente**, sono disponibili quattro sezioni:

1. **Svolgimento test:** Lo studente può selezionare il test da eseguire scegliendo prima il docente. Durante lo svolgimento del test, può inserire una o più risposte per ciascun quesito. Le risposte possono essere aggiornate fino a quando il test non è completato. Lo studente può uscire dal test in corso e continuarlo successivamente. Il test viene considerato "completato" solo dopo che tutte le risposte sono state fornite.
2. **Visualizzazione dei test completati:** Gli studenti possono visualizzare gli esiti dei test svolti e, se abilitato dal docente, visualizzare le risposte corrette per ciascun quesito.
3. **Gestione messaggi:** Lo studente può inviare messaggi a un docente specifico e visualizzare i messaggi ricevuti dai docenti.

4. **Visualizzazione delle statistiche:** Lo studente può accedere alle statistiche e confrontare i risultati.

Ciascun evento che occorre nella piattaforma è stato registrato all'interno di una dedicata collezione **MongoDB**. Il codice per implementare questa funzionalità è stato commentato in quanto non funzionante per tutti i membri del gruppo.



Schermata che mostra la collezione contenente tutti gli eventi di registrazione degli utenti

CODICE SQL

Tabelle

```
DROP DATABASE IF EXISTS PIATTAFORMA_ESQL;
CREATE DATABASE PIATTAFORMA_ESQL;
DROP DATABASE IF EXISTS PROGETTO_BASI;
CREATE DATABASE PROGETTO_BASI;
USE PROGETTO_BASI;

CREATE TABLE IF NOT EXISTS Docente (
    email_docente VARCHAR(255) NOT NULL,
    password VARCHAR(255) NOT NULL,
    nome VARCHAR(50) NOT NULL,
    cognome VARCHAR(50) NOT NULL,
    nome_dipartimento VARCHAR(100) NOT NULL,
    nome_corso VARCHAR(100) NOT NULL,
    PRIMARY KEY (email_docente)
);

CREATE TABLE IF NOT EXISTS Studente (
    email_studente VARCHAR(255) NOT NULL,
    password VARCHAR(255) NOT NULL,
    nome VARCHAR(50) NOT NULL,
    cognome VARCHAR(50) NOT NULL,
    anno_immatricolazione INT NOT NULL,
    codice_alfanumerico VARCHAR(16) UNIQUE NOT NULL,
    PRIMARY KEY (email_studente)
);

CREATE TABLE IF NOT EXISTS Telefono_Docente (
    email_docente VARCHAR(255) NOT NULL,
    rec_telefonico VARCHAR(255) NOT NULL,
    PRIMARY KEY (email_docente, rec_telefonico),
    FOREIGN KEY (email_docente) REFERENCES Docente(email_docente)
);

CREATE TABLE IF NOT EXISTS Telefono_Studente (
    email_studente VARCHAR(255) NOT NULL,
    rec_telefonico VARCHAR(255) NOT NULL,
    PRIMARY KEY (email_studente, rec_telefonico),
    FOREIGN KEY (email_studente) REFERENCES Studente(email_studente)
);

CREATE TABLE IF NOT EXISTS Test (
    titolo VARCHAR(255) NOT NULL,
    data_creazione DATE NOT NULL,
    visualizza_risposte BOOLEAN DEFAULT false,
    email_docente VARCHAR(255) NOT NULL,
    PRIMARY KEY (titolo, email_docente),
    FOREIGN KEY (email_docente) REFERENCES Docente(email_docente)
);

CREATE TABLE IF NOT EXISTS Foto_Test (
    titolo_test VARCHAR(255) NOT NULL,
    email_docente VARCHAR(255) NOT NULL,
    foto BLOB NOT NULL,
    PRIMARY KEY (titolo_test, email_docente),
    FOREIGN KEY (email_docente) REFERENCES Docente(email_docente),
    FOREIGN KEY (titolo_test) REFERENCES Test(titolo)
);

CREATE TABLE IF NOT EXISTS Messaggio_Studente (
    id INT AUTO_INCREMENT,
    data_messaggio DATE NOT NULL,
    titolo VARCHAR(255) NOT NULL,
    testo TEXT NOT NULL,
    email_studente VARCHAR(255) NOT NULL,
    email_docente VARCHAR(255) NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    PRIMARY KEY (id),
```

```

FOREIGN KEY (email_studente) REFERENCES Studente(email_studente),
FOREIGN KEY (email_docente) REFERENCES Docente(email_docente),
FOREIGN KEY (titolo_test) REFERENCES Test(titolo)
);

CREATE TABLE IF NOT EXISTS Messaggio_Docente (
    id INT AUTO_INCREMENT,
    data_messaggio DATE NOT NULL,
    titolo VARCHAR(255) NOT NULL,
    testo TEXT NOT NULL,
    email_docente VARCHAR(255) NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (email_docente) REFERENCES Docente(email_docente),
    FOREIGN KEY (titolo_test) REFERENCES Test(titolo)
);

CREATE TABLE IF NOT EXISTS TabellaSQL (
    nome_tabella VARCHAR(255) NOT NULL,
    data_creazione DATE NOT NULL,
    num_righe INT NOT NULL,
    email_docente VARCHAR(255) NOT NULL,
    PRIMARY KEY (nome_tabella, email_docente),
    FOREIGN KEY (email_docente) REFERENCES Docente(email_docente)
);

CREATE TABLE IF NOT EXISTS Attributo (
    id INT AUTO_INCREMENT,
    nome_tabella VARCHAR(255) NOT NULL,
    nome_attributo VARCHAR(255) NOT NULL,
    tipo VARCHAR(255) NOT NULL,
    chiave BOOLEAN DEFAULT false,
    PRIMARY KEY (id),
    FOREIGN KEY (nome_tabella) REFERENCES TabellaSQL(nome_tabella)
);

CREATE TABLE IF NOT EXISTS Vincolo_Di_Integrita (
    id_attributo_referenziante INT NOT NULL,
    id_attributo_referenziato INT NOT NULL,
    PRIMARY KEY (id_attributo_referenziante, id_attributo_referenziato),
    FOREIGN KEY (id_attributo_referenziante) REFERENCES Attributo(id),
    FOREIGN KEY (id_attributo_referenziato) REFERENCES Attributo(id)
);

CREATE TABLE IF NOT EXISTS Studente_Svolge_Test (
    data_ultima_risposta DATE NOT NULL,
    data_prima_risposta DATE NOT NULL,
    email_studente VARCHAR(255) NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    stato ENUM('Aperto', 'InCompletamento', 'Concluso') NOT NULL,
    PRIMARY KEY (email_studente, titolo_test),
    FOREIGN KEY (email_studente) REFERENCES Studente(email_studente),
    FOREIGN KEY (titolo_test) REFERENCES Test(titolo)
);

CREATE TABLE IF NOT EXISTS Quesito (
    numero_progressivo INT NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    difficolta ENUM('Basso', 'Medio', 'Alto') NOT NULL,
    descrizione VARCHAR(255) NOT NULL,
    tipo ENUM('Chiuso', 'Codice') NOT NULL,
    num_risposte INT NOT NULL,
    PRIMARY KEY (numero_progressivo, titolo_test),
    FOREIGN KEY (titolo_test) REFERENCES Test(titolo)
);

CREATE TABLE IF NOT EXISTS Quesito_Riferisce_Tabella (
    numero_quesito INT NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    nome_tabella_sql VARCHAR(255) NOT NULL,
    PRIMARY KEY (numero_quesito, titolo_test, nome_tabella_sql),
    FOREIGN KEY (numero_quesito, titolo_test) REFERENCES Quesito(numero_progressivo,
titolo_test),
    FOREIGN KEY (nome_tabella_sql) REFERENCES TabellaSQL(nome_tabella)
);

```

```

CREATE TABLE IF NOT EXISTS Opzione (
    numerazione INT NOT NULL,
    numero_quesito INT NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    campo_testo VARCHAR(255) NOT NULL,
    corretta BOOLEAN NOT NULL,
    PRIMARY KEY (numerazione, numero_quesito, titolo_test),
    FOREIGN KEY (numero_quesito, titolo_test) REFERENCES Quesito(numero_progressivo,
titolo_test)
);

CREATE TABLE IF NOT EXISTS RispostaChiusa (
    id INT AUTO_INCREMENT,
    esito BOOLEAN NOT NULL,
    email_studente VARCHAR(255),
    numero_quesito INT NOT NULL,
    numero_opzione INT NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (email_studente) REFERENCES Studente(email_studente),
    FOREIGN KEY (numero_quesito, titolo_test) REFERENCES Quesito(numero_progressivo,
titolo_test),
    FOREIGN KEY (numero_opzione, numero_quesito, titolo_test) REFERENCES Opzione(numerazione,
numero_quesito, titolo_test)
);

CREATE TABLE IF NOT EXISTS RispostaCodice (
    id INT AUTO_INCREMENT,
    esito BOOLEAN NOT NULL,
    email_studente VARCHAR(255),
    testoSQL TEXT NOT NULL,
    numero_quesito INT NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (email_studente) REFERENCES Studente(email_studente),
    FOREIGN KEY (numero_quesito, titolo_test) REFERENCES Quesito(numero_progressivo,
titolo_test)
);

CREATE TABLE IF NOT EXISTS Soluzione_Codice (
    numerazione INT NOT NULL,
    numero_quesito INT NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    codice TEXT NOT NULL,
    PRIMARY KEY (numerazione, numero_quesito, titolo_test),
    FOREIGN KEY (numero_quesito, titolo_test) REFERENCES Quesito(numero_progressivo,
titolo_test)
);

CREATE TABLE IF NOT EXISTS TestCompleti(
    email_studente VARCHAR(255) NOT NULL,
    titolo_test VARCHAR(255) NOT NULL,
    punteggio INT NOT NULL,
    totale INT NOT NULL,
    PRIMARY KEY (email_studente, titolo_test),
    FOREIGN KEY (email_studente) REFERENCES Studente(email_studente),
    FOREIGN KEY (titolo_test) REFERENCES Test(titolo)
);

-- usata per il trigger di num_righe
CREATE TABLE IF NOT EXISTS TabellaAppoggio (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome_tabella VARCHAR(255) NOT NULL,
    FOREIGN KEY(nome_tabella) REFERENCES TabellaSQL(nome_tabella)
);

DROP VIEW IF EXISTS Classifica_Studenti_Test_Completati;
CREATE VIEW Classifica_Studenti_Test_Completati AS
SELECT S.codice_alfanumerico, COUNT(*) AS NumeroTestCompleti
FROM Studente_Svolge_Test SST
JOIN Studente S ON SST.email_studente = S.email_studente
WHERE SST.stato = 'Concluso'
GROUP BY S.codice_alfanumerico

```

```

ORDER BY NumeroTestCompletati DESC;

DROP VIEW IF EXISTS Classifica_Studenti_Accuratezza;
CREATE VIEW Classifica_Studenti_Accuratezza AS
SELECT
    S.codice_alfanumerico,
    SUM(CASE WHEN Risposte.esito = TRUE THEN 1 ELSE 0 END) AS RisposteCorrette,
    COUNT(*) AS TotaleRisposte,
    (SUM(CASE WHEN Risposte.esito = TRUE THEN 1 ELSE 0 END) / COUNT(*)) * 100 AS
PercentualeCorrettezza
FROM (
    SELECT email_studente, esito FROM RispostaChiusa
    UNION ALL
    SELECT email_studente, esito FROM RispostaCodice
) AS Risposte
JOIN Studente S ON Risposte.email_studente = S.email_studente
GROUP BY
    S.codice_alfanumerico
ORDER BY
    PercentualeCorrettezza DESC;

DROP VIEW IF EXISTS Classifica_Quesiti_Risposte;
CREATE VIEW Classifica_Quesiti_Risposte AS
SELECT
    Q.numero_progressivo,
    Q.titolo_test,
    Q.descrizione,
    Q.difficolta,
    Q.tipo,
    (IFNULL(RC.NumRisposteChiuse, 0) + IFNULL(RS.NumRisposteCodice, 0)) AS
TotaleRisposteInserite
FROM
    Quesito Q
LEFT JOIN
    (SELECT
        numero_quesito,
        titolo_test,
        COUNT(*) AS NumRisposteChiuse
    FROM
        RispostaChiusa
    GROUP BY
        numero_quesito, titolo_test
    ) RC ON Q.numero_progressivo = RC.numero_quesito AND Q.titolo_test = RC.titolo_test
LEFT JOIN
    (SELECT
        numero_quesito,
        titolo_test,
        COUNT(*) AS NumRisposteCodice
    FROM
        RispostaCodice
    GROUP BY
        numero_quesito, titolo_test
    ) RS ON Q.numero_progressivo = RS.numero_quesito AND Q.titolo_test = RS.titolo_test
ORDER BY
    TotaleRisposteInserite DESC;

```

Stored procedure

```

DROP PROCEDURE IF EXISTS REGISTRAZIONE_DOCENTE;
DELIMITER //
CREATE PROCEDURE REGISTRAZIONE_DOCENTE(
    IN p_email_docente VARCHAR(255),
    IN p_password VARCHAR(255),
    IN p_nome VARCHAR(255),
    IN p_cognome VARCHAR(255),
    IN p_nome_dipartimento VARCHAR(255),
    IN p_nome_corso VARCHAR(255),
    IN p_rec_telefonico VARCHAR(255),
    OUT p_success BOOLEAN
)
BEGIN

```



```

DECLARE email_count_docente INT;
DECLARE email_count_studente INT;

SELECT COUNT(*) INTO email_count_docente FROM Docente WHERE email_docente =
p_email_docente;

SELECT COUNT(*) INTO email_count_studente FROM Studente WHERE email_studente =
p_email_docente;

IF email_count_docente > 0 OR email_count_studente > 0 THEN
    SET p_success = FALSE;
ELSE
    SET p_password = SHA2(p_password, 256);

    INSERT INTO Docente(email_docente, password, nome, cognome, nome_dipartimento,
nome_corso)
VALUES (p_email_docente, p_password, p_nome, p_cognome, p_nome_dipartimento,
p_nome_corso);

    IF p_rec_telefonico IS NOT NULL THEN
        INSERT INTO Telefono_Docente(email_docente, rec_telefonico)
VALUES (p_email_docente, p_rec_telefonico);
    END IF;

    SET p_success = TRUE;
END IF;
END//
DELIMITER;

DROP PROCEDURE IF EXISTS REGISTRAZIONE_STUDENTE;
DELIMITER //
CREATE PROCEDURE REGISTRAZIONE_STUDENTE(
    IN p_email_studente VARCHAR(255),
    IN p_password VARCHAR(255),
    IN p_nome VARCHAR(255),
    IN p_cognome VARCHAR(255),
    IN p_anno_immatricolazione INT,
    IN p_codice_alfanumerico VARCHAR(16),
    IN p_rec_telefonico VARCHAR(255),
    OUT p_success BOOLEAN
)
BEGIN
    DECLARE email_count_studente INT;
    DECLARE email_count_docente INT;
    DECLARE code_count INT;

    SELECT COUNT(*) INTO email_count_studente FROM Studente WHERE email_studente =
p_email_studente;

    SELECT COUNT(*) INTO email_count_docente FROM Docente WHERE email_docente =
p_email_studente;

    SELECT COUNT(*) INTO code_count FROM Studente WHERE codice_alfanumerico =
p_codice_alfanumerico;

    IF (email_count_studente > 0 OR email_count_docente > 0 OR code_count > 0) THEN
        SET p_success = FALSE;
    ELSE
        SET p_password = SHA2(p_password, 256);

        INSERT INTO Studente(email_studente, password, nome, cognome, anno_immatricolazione,
codice_alfanumerico)
VALUES (p_email_studente, p_password, p_nome, p_cognome, p_anno_immatricolazione,
p_codice_alfanumerico);

        IF p_rec_telefonico IS NOT NULL THEN
            INSERT INTO Telefono_Studente(email_studente, rec_telefonico)
VALUES (p_email_studente, p_rec_telefonico);
        END IF;

        SET p_success = TRUE;
    END IF;
END//
DELIMITER;

```

```

DROP PROCEDURE IF EXISTS AUTENTICAZIONE_UTENTE;
DELIMITER //
CREATE PROCEDURE AUTENTICAZIONE_UTENTE(
    IN p_email VARCHAR(255),
    IN p_password VARCHAR(255),
    OUT p_status INT,
    OUT p_user_type VARCHAR(10),
    OUT p_nome VARCHAR(255)
)
BEGIN
    SET p_status = 0;
    SET p_user_type = 'nessuno';

    IF EXISTS(SELECT * FROM Studente WHERE email_studente = p_email) THEN
        SET p_status = 1;
        IF EXISTS(SELECT * FROM Studente WHERE email_studente = p_email AND password =
SHA2(p_password, 256)) THEN
            SET p_status = 2;
            SET p_user_type = 'studente';
            SELECT nome INTO p_nome FROM Studente WHERE email_studente = p_email AND password =
SHA2(p_password, 256);
        END IF;
    END IF;

    IF EXISTS(SELECT * FROM Docente WHERE email_docente = p_email) THEN
        SET p_status = 1;
        IF EXISTS(SELECT * FROM Docente WHERE email_docente = p_email AND password =
SHA2(p_password, 256)) THEN
            SET p_status = 2;
            SET p_user_type = 'docente';
            SELECT nome INTO p_nome FROM Docente WHERE email_docente = p_email AND password =
SHA2(p_password, 256);
        END IF;
    END IF;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS CREAZIONE_TABELLA;
DELIMITER //
CREATE PROCEDURE CREAZIONE_TABELLA(
    IN p_nome_tabella VARCHAR(225),
    IN p_email_docente VARCHAR(255)
)
BEGIN
    INSERT INTO TabellaSQL(nome_tabella, data_creazione, num_righe, email_docente)
    VALUES (p_nome_tabella, CURRENT_DATE(), '0', p_email_docente);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS NUOVO_ATTRIBUTO;
DELIMITER //
CREATE PROCEDURE NUOVO_ATTRIBUTO(
    IN p_nome_tabella VARCHAR(255),
    IN p_nome_attributo VARCHAR(255),
    IN p_tipo VARCHAR(255),
    IN p_chiave BOOLEAN
)
BEGIN
    INSERT INTO Attributo(nome_tabella, nome_attributo, tipo, chiave)
    VALUES (p_nome_tabella, p_nome_attributo, p_tipo, p_chiave);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS NUOVO_VINCOLO;
DELIMITER //
CREATE PROCEDURE NUOVO_VINCOLO(
    IN p_nome_attributo1 VARCHAR(255),
    IN p_nome_attributo2 VARCHAR(255),
    IN p_nome_tabella1 VARCHAR(255),
    IN p_nome_tabella2 VARCHAR(255)
)
BEGIN
    DECLARE id_1 INT;
    DECLARE id_2 INT;

    SELECT id INTO id_1
    FROM Attributo

```

```

WHERE nome_attributo = p_nome_attributo1 AND nome_tabella = p_nome_tabella1;

SELECT id INTO id_2
FROM Attributo
WHERE nome_attributo = p_nome_attributo2 AND nome_tabella = p_nome_tabella2;

INSERT INTO Vincolo_Di_Integrita(id_attributo_referenziante, id_attributo_referenziato)
VALUES (id_1, id_2);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZAZIONE_TABELLA;
DELIMITER //
CREATE PROCEDURE VISUALIZZAZIONE_TABELLA(
    IN p_email_docente VARCHAR(255)
)
BEGIN
    SELECT nome_tabella, data_creazione, num_righe
    FROM TabellaSQL
    WHERE email_docente = p_email_docente;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS NUOVO_TEST;
DELIMITER //
CREATE PROCEDURE NUOVO_TEST(
    IN p_titolo VARCHAR(225),
    IN p_data_creazione DATE,
    IN p_email_docente VARCHAR(255)
)
BEGIN
    INSERT INTO Test(titolo, data_creazione, email_docente)
    VALUES (p_titolo, p_data_creazione, p_email_docente);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERISCI_QUESITO;
DELIMITER //
CREATE PROCEDURE INSERISCI_QUESITO (
    IN p_titolo_test VARCHAR(255),
    IN p_difficolta ENUM('Basso', 'Medio', 'Alto'),
    IN p_descrizione VARCHAR(255),
    IN p_tipo ENUM('Chiuso', 'Codice'),
    OUT p_numero_progressivo INT
)
BEGIN
    INSERT INTO Quesito (titolo_test, difficolta, descrizione, tipo, num_risposte)
    VALUES (p_titolo_test, p_difficolta, p_descrizione, p_tipo, '0');

    SELECT MAX(numero_progressivo) INTO p_numero_progressivo
    FROM Quesito
    WHERE titolo_test = p_titolo_test;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERISCI_FOTO_TEST;
DELIMITER //
CREATE PROCEDURE INSERISCI_FOTO_TEST (
    IN p_titolo_test VARCHAR(255),
    IN p_email_docente VARCHAR(255),
    IN p_foto BLOB
)
BEGIN
    INSERT INTO Foto_test (titolo_test, email_docente, foto)
    VALUES (p_titolo_test, p_email_docente, p_foto);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERISCI_QUESITO_TABELLA;
DELIMITER //
CREATE PROCEDURE INSERISCI_QUESITO_TABELLA(
    IN p_numero_quesito INT,
    IN p_titolo_test VARCHAR(255),
    IN p_nome_tabella_sql VARCHAR(255)
)
BEGIN
    INSERT INTO Quesito_Riferisce_Tabella(numero_quesito, titolo_test, nome_tabella_sql)

```

```

VALUES (p_numero_quesito, p_titolo_test, p_nome_tabella_sql);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERISCI_OPZIONE;
DELIMITER //
CREATE PROCEDURE INSERISCI_OPZIONE(
    IN p_numero_quesito INT,
    IN p_titolo_test VARCHAR(255),
    IN p_campo_testo VARCHAR(255),
    IN p_corretta BOOLEAN
)
BEGIN
    INSERT INTO Opzione (numero_quesito, titolo_test, campo_testo, corretta)
    VALUES (p_numero_quesito, p_titolo_test, p_campo_testo, p_corretta);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERISCI_SOLUZIONE_CODICE;
DELIMITER //
CREATE PROCEDURE INSERISCI_SOLUZIONE_CODICE(
    IN p_numero_quesito INT,
    IN p_titolo_test VARCHAR(255),
    IN p_codice TEXT
)
BEGIN
    INSERT INTO Soluzione_Codice (numero_quesito, titolo_test, codice)
    VALUES (p_numero_quesito, p_titolo_test, p_codice);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZAZIONE_TEST;
DELIMITER //
CREATE PROCEDURE VISUALIZZAZIONE_TEST(
    IN p_email_docente VARCHAR(255)
)
BEGIN
    SELECT titolo, data_creazione, visualizza_risposte
    FROM Test
    WHERE email_docente = p_email_docente;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZAZIONE_QUESITI;
DELIMITER //
CREATE PROCEDURE VISUALIZZAZIONE_QUESITI(
    IN p_nome_test VARCHAR(255)
)
BEGIN
    SELECT *
    FROM Quesito
    WHERE p_nome_test = titolo_test;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZAZIONE_QUESITO_TABELLA;
DELIMITER //
CREATE PROCEDURE VISUALIZZAZIONE_QUESITO_TABELLA(
    IN p_numero_quesito INT,
    IN p_titolo_test VARCHAR(255)
)
BEGIN
    SELECT *
    FROM Quesito_Riferisce_Tabella
    WHERE p_numero_quesito = numero_quesito AND p_titolo_test = titolo_test;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZAZIONE_OPZIONI;
DELIMITER //
CREATE PROCEDURE VISUALIZZAZIONE_OPZIONI(
    IN p_titolo_test VARCHAR(255),
    IN p_numero_quesito INT
)
BEGIN
    SELECT numerazione, campo_testo, corretta
    FROM Opzione

```

```

WHERE p_titolo_test = titolo_test AND p_numero_quesito = numero_quesito;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZAZIONE_SOLUZIONE_CODICE;
DELIMITER //
CREATE PROCEDURE VISUALIZZAZIONE_SOLUZIONE_CODICE(
    IN p_titolo_test VARCHAR(255),
    IN p_numero_quesito INT
)
BEGIN
    SELECT numerazione, codice
    FROM Soluzione_Codice
    WHERE titolo_test = p_titolo_test AND numero_quesito = p_numero_quesito;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS MODIFICA_VISUALIZZA_RISPOSTE;
DELIMITER //
CREATE PROCEDURE MODIFICA_VISUALIZZA_RISPOSTE(
    IN p_titolo_test VARCHAR(255),
    IN p_nuovo_valore BOOLEAN
)
BEGIN
    UPDATE Test
    SET visualizza_risposte = p_nuovo_valore
    WHERE titolo = p_titolo_test;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERIMENTO_MESSAGGIO_DOCENTE;
DELIMITER //
CREATE PROCEDURE INSERIMENTO_MESSAGGIO_DOCENTE(
    IN p_data_messaggio DATE,
    IN p_titolo VARCHAR(225),
    IN p_testo TEXT,
    IN p_email_docente VARCHAR(255),
    IN p_titolo_test VARCHAR(255)
)
BEGIN
    INSERT INTO Messaggio_Docente(data_messaggio, titolo, testo, email_docente, titolo_test)
    VALUES (p_data_messaggio, p_titolo, p_testo, p_email_docente, p_titolo_test);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERIMENTO_MESSAGGIO_STUDENTE;
DELIMITER //
CREATE PROCEDURE INSERIMENTO_MESSAGGIO_STUDENTE(
    IN p_data_messaggio DATE,
    IN p_titolo VARCHAR(225),
    IN p_testo TEXT,
    IN p_email_studente VARCHAR(255),
    IN p_email_docente VARCHAR(255),
    IN p_titolo_test VARCHAR(255)
)
BEGIN
    INSERT INTO Messaggio_Studente(data_messaggio, titolo, testo, email_studente,
    email_docente, titolo_test)
    VALUES (p_data_messaggio, p_titolo, p_testo, p_email_studente, p_email_docente,
    p_titolo_test);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZA_MESSAGGI_INVIATI_DOCENTE;
DELIMITER //
CREATE PROCEDURE VISUALIZZA_MESSAGGI_INVIATI_DOCENTE(
    IN p_email_docente VARCHAR(255)
)
BEGIN
    SELECT *
    FROM Messaggio_Docente
    WHERE email_docente = p_email_docente;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZA_MESSAGGI_RICEVUTI_DOCENTE;

```

```

DELIMITER //
CREATE PROCEDURE VISUALIZZA_MESSAGGI_RICEVUTI_DOCENTE (
    IN p_email_docente VARCHAR(255)
)
BEGIN
    SELECT *
    FROM Messaggio_Studente
    WHERE email_docente = p_email_docente;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZA_MESSAGGI_INVIATI_STUDENTE;
DELIMITER //
CREATE PROCEDURE VISUALIZZA_MESSAGGI_INVIATI_STUDENTE (
    IN p_email_studente VARCHAR(255)
)
BEGIN
    SELECT *
    FROM Messaggio_Studente
    WHERE email_studente = p_email_studente;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZA_MESSAGGI_RICEVUTI_STUDENTE;
DELIMITER //
CREATE PROCEDURE VISUALIZZA_MESSAGGI_RICEVUTI_STUDENTE (
    IN p_email_studente VARCHAR(255)
)
BEGIN
    SELECT titolo, testo, data_messaggio, email_docente, titolo_test
    FROM Messaggio_Docente;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERISCI_O_AGGIORNA_RISPOSTA_CHIUSA;
DELIMITER //
CREATE PROCEDURE INSERISCI_O_AGGIORNA_RISPOSTA_CHIUSA (
    IN p_esito BOOLEAN,
    IN p_email_studente VARCHAR(255),
    IN p_numero_quesito INT,
    IN p_numero_opzione INT,
    IN p_titolo_test VARCHAR(255)
)
BEGIN
    DECLARE existing_id INT;

    SELECT id INTO existing_id FROM RispostaChiusa
    WHERE email_studente = p_email_studente
    AND numero_quesito = p_numero_quesito
    AND titolo_test = p_titolo_test;

    IF existing_id IS NOT NULL THEN
        UPDATE RispostaChiusa
        SET esito = p_esito, numero_opzione = p_numero_opzione
        WHERE id = existing_id;
    ELSE
        INSERT INTO RispostaChiusa (esito, email_studente, numero_quesito, numero_opzione,
titolo_test)
        VALUES (p_esito, p_email_studente, p_numero_quesito, p_numero_opzione,
p_titolo_test);
    END IF;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VERIFICA_OPZIONE_CORRETTA;
DELIMITER //
CREATE PROCEDURE VERIFICA_OPZIONE_CORRETTA (
    IN p_numero_quesito INT,
    IN p_numero_opzione INT,
    IN p_titolo_test VARCHAR(255),
    OUT p_corretta BOOLEAN
)
BEGIN
    SELECT corretta INTO p_corretta
    FROM Opzione
    WHERE numero_quesito = p_numero_quesito
    AND numerazione = p_numero_opzione

```

```

        AND titolo_test = p_titolo_test;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERISCI_O_AGGIORNA_RISPOSTA_CODICE;
DELIMITER //
CREATE PROCEDURE INSERISCI_O_AGGIORNA_RISPOSTA_CODICE(
    IN p_esito BOOLEAN,
    IN p_emailStudente VARCHAR(255),
    IN p_numeroQuesito INT,
    IN p_sqlTesto TEXT,
    IN p_titoloTest VARCHAR(255)
)
BEGIN
    DECLARE rispostaEsistente INT;

    SELECT id INTO rispostaEsistente
    FROM RispostaCodice
    WHERE email_studente = p_emailStudente AND numero_quesito = p_numeroQuesito AND
    titolo_test = p_titoloTest;

    IF rispostaEsistente IS NOT NULL THEN
        UPDATE RispostaCodice
        SET esito = p_esito, testoSQL = p_sqlTesto
        WHERE id = rispostaEsistente;
    ELSE
        INSERT INTO RispostaCodice (esito, email_studente, numero_quesito, testoSQL,
        titolo_test)
        VALUES (p_esito, p_emailStudente, p_numeroQuesito, p_sqlTesto, p_titoloTest);
    END IF;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS INSERISCI_SVOLGIMENTO_TEST;
DELIMITER //
CREATE PROCEDURE INSERISCI_SVOLGIMENTO_TEST(
    IN p_data_ultima_risposta DATE,
    IN p_data_prima_risposta DATE,
    IN p_email_studente VARCHAR(255),
    IN p_titolo_test VARCHAR(255),
    IN p_stato ENUM('Aperto', 'InCompletamento', 'Concluso')
)
BEGIN
    INSERT INTO Studente_Svolge_Test(data_ultima_risposta, data_prima_risposta,
    email_studente, titolo_test, stato)
    VALUES (p_data_ultima_risposta, p_data_prima_risposta, p_email_studente, p_titolo_test,
    p_stato);
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS CALCOLA_ESITO_TEST;
DELIMITER //
CREATE PROCEDURE CALCOLA_ESITO_TEST(
    IN p_email_studente VARCHAR(255),
    IN p_titolo_test VARCHAR(255)
)
BEGIN
    DECLARE p_quesiti_totali INT DEFAULT 0;
    DECLARE p_risposte_chiuse_corrette INT DEFAULT 0;
    DECLARE p_risposte_codice_corrette INT DEFAULT 0;
    DECLARE p_quesiti_corretti INT DEFAULT 0;

    SELECT COUNT(*) INTO p_quesiti_totali
    FROM Quesito
    WHERE titolo_test = p_titolo_test;

    SELECT COUNT(*) INTO p_risposte_chiuse_corrette
    FROM RispostaChiusa
    WHERE email_studente = p_email_studente
        AND titolo_test = p_titolo_test
        AND esito = TRUE;

    SELECT COUNT(*) INTO p_risposte_codice_corrette
    FROM RispostaCodice
    WHERE email_studente = p_email_studente
        AND titolo_test = p_titolo_test

```

```

        AND esito = TRUE;

SET p_quesiti_corretti = p_risposte_chiuse_corrette + p_risposte_codice_corrette;

INSERT INTO TestCompleti (email_studente, titolo_test, punteggio, totale)
VALUES (p_email_studente, p_titolo_test, p_quesiti_corretti, p_quesiti_totali)
ON DUPLICATE KEY UPDATE
    punteggio = p_quesiti_corretti,
    totale = p_quesiti_totali;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZA_TEST_NON_COMPLETI;
DELIMITER //
CREATE PROCEDURE VISUALIZZA_TEST_NON_COMPLETI(
    IN p_email_docente VARCHAR(255),
    IN p_email_studente VARCHAR(255)
)
BEGIN
    SELECT t.titolo, t.data_creazione
    FROM test t
    LEFT JOIN studente_svolge_test sst
        ON t.titolo = sst.titolo_test
        AND sst.email_studente = p_email_studente
        AND sst.stato = 'Concluso'
    WHERE t.email_docente = p_email_docente
    AND sst.titolo_test IS NULL;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS AGGIORNA_DATI_TEST;
DELIMITER //
CREATE PROCEDURE AGGIORNA_DATI_TEST(
    IN p_email_studente VARCHAR(255),
    IN p_titolo_test VARCHAR(255)
)
BEGIN
    DECLARE v_exists INT;

    SELECT COUNT(*) INTO v_exists
    FROM Studente_Svolge_Test
    WHERE email_studente = p_email_studente AND titolo_test = p_titolo_test;

    IF v_exists = 0 THEN
        INSERT INTO Studente_Svolge_Test (email_studente, titolo_test, data_prima_risposta,
        data_ultima_risposta, stato)
        VALUES (p_email_studente, p_titolo_test, NOW(), NOW(), 'InCompletamento');
    ELSE
        UPDATE Studente_Svolge_Test
        SET data_ultima_risposta = NOW()
        WHERE email_studente = p_email_studente AND titolo_test = p_titolo_test;
    END IF;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZA_ESITI;
DELIMITER //
CREATE PROCEDURE VISUALIZZA_ESITI(
    IN p_email_studente VARCHAR(255)
)
BEGIN
    SELECT *
    FROM TestCompleti
    WHERE email_studente = p_email_studente;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZA_OPZIONE_SCELTA;
DELIMITER //
CREATE PROCEDURE VISUALIZZA_OPZIONE_SCELTA(
    IN p_email_studente VARCHAR(255),
    IN p_titolo_test VARCHAR(255),
    IN p_numero_quesito INT,

```



```

        OUT p_campo_testo VARCHAR(255)
    )
BEGIN
    DECLARE numero_opzione_scelta INT;

    SELECT numero_opzione INTO numero_opzione_scelta
    FROM RispostaChiusa
    WHERE email_studente = p_email_studente
        AND titolo_test = p_titolo_test
        AND numero_quesito = p_numero_quesito;

    SELECT campo_testo INTO p_campo_testo
    FROM Opzione
    WHERE titolo_test = p_titolo_test
        AND numero_quesito = p_numero_quesito
        AND numerazione = numero_opzione_scelta;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS VISUALIZZA_TESTO_INSERTITO;
DELIMITER //
CREATE PROCEDURE VISUALIZZA_TESTO_INSERTITO(
    IN p_email_studente VARCHAR(255),
    IN p_titolo_test VARCHAR(255),
    IN p_numero_quesito INT,
    OUT p_testoSQL VARCHAR(255)
)
BEGIN
    SELECT testoSQL INTO p_testoSQL
    FROM RispostaCodice
    WHERE titolo_test = p_titolo_test
        AND numero_quesito = p_numero_quesito
        AND email_studente = p_email_studente;
END//
DELIMITER ;

DROP PROCEDURE IF EXISTS AGGIORNA_VISUALIZZAZIONE_RISPOSTE;
DELIMITER //
CREATE PROCEDURE AGGIORNA_VISUALIZZAZIONE_RISPOSTE(
    IN p_titolo_test VARCHAR(255),
    IN p_email_docente VARCHAR(255),
    IN p_new_state BOOLEAN
)
BEGIN
    UPDATE Test
    SET visualizza_risposte = p_new_state
    WHERE titolo = p_titolo_test AND email_docente = p_email_docente;
END//
DELIMITER ;

```

Triggers

```

DELIMITER //

CREATE TRIGGER set_numero_progressivo_quesito
BEFORE INSERT ON Quesito
FOR EACH ROW
BEGIN
    DECLARE max_numero INT;

    SELECT IFNULL(MAX(numero_progressivo), 0) INTO max_numero
    FROM Quesito
    WHERE titolo_test = NEW.titolo_test;

    SET NEW.numero_progressivo = max_numero + 1;
END;
//

CREATE TRIGGER set_numero_progressivo_opzione
BEFORE INSERT ON Opzione
FOR EACH ROW

```

```

BEGIN
    DECLARE max_numero INT;

    SELECT IFNULL(MAX(enumerazione), 0) INTO max_numero
    FROM Opzione
    WHERE numero_quesito = NEW.numero_quesito
        AND titolo_test = NEW.titolo_test;

    SET NEW.enumerazione = max_numero + 1;
END;
//

CREATE TRIGGER set_numero_progressivo_soluzione_codice
BEFORE INSERT ON Soluzione_Codice
FOR EACH ROW
BEGIN
    DECLARE max_numero INT;

    SELECT IFNULL(MAX(enumerazione), 0) INTO max_numero
    FROM Soluzione_Codice
    WHERE numero_quesito = NEW.numero_quesito
        AND titolo_test = NEW.titolo_test;

    SET NEW.enumerazione = max_numero + 1;
END;
//

CREATE TRIGGER cambio_stato_test_concluso_docente
AFTER UPDATE ON Test
FOR EACH ROW
BEGIN
    DECLARE nstudentesvolgetest INT;
    IF NEW.visualizza_risposte = TRUE THEN
        UPDATE studente_svolge_test
        SET stato = 'concluso'
        WHERE titolo_test = NEW.titolo;
    END IF;
END;
//

CREATE TRIGGER incrementa_num_righe
AFTER INSERT ON TabellaAppoggio
FOR EACH ROW
BEGIN
    DECLARE tabella_nome VARCHAR(255);

    SET tabella_nome = NEW.nome_tabella;

    UPDATE TabellaSQL
    SET num_righe = num_righe + 1
    WHERE nome_tabella = tabella_nome;
END;
//

CREATE TRIGGER tr_after_insert_risposta_chiusa
AFTER INSERT ON RispostaChiusa
FOR EACH ROW
BEGIN
    DECLARE total_questions INT;
    DECLARE answered_chiusa INT;
    DECLARE answered_codice INT;
    DECLARE answered_questions INT;

    CALL AGGIORNA_DATI_TEST(NEW.email_studente, NEW.titolo_test);

    SELECT COUNT(*) INTO total_questions
    FROM Quesito
    WHERE titolo_test = NEW.titolo_test;

    SELECT COUNT(*) INTO answered_chiusa
    FROM RispostaChiusa
    WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

    SELECT COUNT(*) INTO answered_codice
    FROM RispostaCodice
    WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

```

```

        SET answered_questions = answered_chiusa + answered_codice;

        IF answered_questions = total_questions THEN
            UPDATE Studente_Svolge_Test
            SET stato = 'Concluso'
            WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

            CALL CALCOLA_ESITO_TEST(NEW.email_studente, NEW.titolo_test);
        END IF;
    END;
//

CREATE TRIGGER tr_after_insert_risposta_codice
AFTER INSERT ON RispostaCodice
FOR EACH ROW
BEGIN
    DECLARE total_questions INT;
    DECLARE answered_chiusa INT;
    DECLARE answered_codice INT;
    DECLARE answered_questions INT;

    CALL AGGIORNA_DATI_TEST(NEW.email_studente, NEW.titolo_test);

    SELECT COUNT(*) INTO total_questions
    FROM Quesito
    WHERE titolo_test = NEW.titolo_test;

    SELECT COUNT(*) INTO answered_chiusa
    FROM RispostaChiusa
    WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

    SELECT COUNT(*) INTO answered_codice
    FROM RispostaCodice
    WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

    SET answered_questions = answered_chiusa + answered_codice;

    IF answered_questions = total_questions THEN
        UPDATE Studente_Svolge_Test
        SET stato = 'Concluso'
        WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

        CALL CALCOLA_ESITO_TEST(NEW.email_studente, NEW.titolo_test);
    END IF;
END;
//

CREATE TRIGGER after_insert_risposta
AFTER INSERT ON RispostaChiusa
FOR EACH ROW
BEGIN
    UPDATE Quesito
    SET num_risposte = num_risposte + 1
    WHERE numero_progressivo = NEW.numero_quesito
    AND titolo_test = NEW.titolo_test;
END;
//

CREATE TRIGGER after_insert_risposta_codice
AFTER INSERT ON RispostaCodice
FOR EACH ROW
BEGIN
    UPDATE Quesito
    SET num_risposte = num_risposte + 1
    WHERE numero_progressivo = NEW.numero_quesito
    AND titolo_test = NEW.titolo_test;
END;
//

CREATE TRIGGER tr_after_update_risposta_chiusa
AFTER UPDATE ON RispostaChiusa
FOR EACH ROW
BEGIN
    DECLARE total_questions INT;
    DECLARE answered_chiusa INT;
    DECLARE answered_codice INT;
    DECLARE answered_questions INT;

```

```

SELECT COUNT(*) INTO total_questions
FROM Quesito
WHERE titolo_test = NEW.titolo_test;

SELECT COUNT(*) INTO answered_chiusa
FROM RispostaChiusa
WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

SELECT COUNT(*) INTO answered_codice
FROM RispostaCodice
WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

SET answered_questions = answered_chiusa + answered_codice;

IF answered_questions = total_questions THEN
    UPDATE Studente_Svolge_Test
    SET stato = 'Concluso'
    WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

    CALL CALCOLA_ESITO_TEST(NEW.email_studente, NEW.titolo_test);
END IF;
END;
//

CREATE TRIGGER tr_after_update_risposta_codice
AFTER UPDATE ON RispostaCodice
FOR EACH ROW
BEGIN
    DECLARE total_questions INT;
    DECLARE answered_questions INT;
    DECLARE answered_chiusa INT;
    DECLARE answered_codice INT;

    SELECT COUNT(*) INTO total_questions
    FROM Quesito
    WHERE titolo_test = NEW.titolo_test;

    SELECT COUNT(*) INTO answered_chiusa
    FROM RispostaChiusa
    WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

    SELECT COUNT(*) INTO answered_codice
    FROM RispostaCodice
    WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

    SET answered_questions = answered_chiusa + answered_codice;

    IF answered_questions = total_questions THEN
        UPDATE Studente_Svolge_Test
        SET stato = 'Concluso'
        WHERE email_studente = NEW.email_studente AND titolo_test = NEW.titolo_test;

        CALL CALCOLA_ESITO_TEST(NEW.email_studente, NEW.titolo_test);
    END IF;
END;
//

CREATE TRIGGER tr_after_update_visualizza_risposte
AFTER UPDATE ON Test
FOR EACH ROW
BEGIN
    IF OLD.visualizza_risposte = FALSE AND NEW.visualizza_risposte = TRUE THEN
        UPDATE Studente_Svolge_Test
        SET stato = 'Concluso'
        WHERE titolo_test = NEW.titolo_test;
    END IF;
END;
//

DELIMITER ;

```